Mês 11 e 12 — Submissão, Relatório Final e Encerramento (jul–ago/2027)

Guia de Estudos - PIBITI

Detecção de *Deepfakes* de Áudio usando *Wavelet Packet Transform* (WPT)

20 de setembro de 2026

Sumário

I MÊS 11: Submissão do Artigo e Congelamento da Biblioteca **3**

1 O que o plano exige (Mês 11) **4**

2 Revisão do Artigo **5**
2.1 Checklist de Revisão 5
2.2 Automatizando Tabelas LaTeX com Python 5

3 Eventos Alvo **7**

4 Congelar Biblioteca **8**

5 LICENSE **9**
5.1 Licença MIT 9

6 CITATION.cff **10**

7 DOI via Zenodo **11**

8 Licenças e LGPD **12**

II MÊS 12: Relatório Final e Apresentação **13**

9 O que o plano exige (Mês 12) **14**

10 Relatório Final **15**
10.1 Estrutura / Template do Relatório 15

11 Compilar Resultados Finais **16**

12 Apresentação / Poster **18**

13 Entregas Finais **19**

14 Trabalhos Futuros **20**
14.1 Cross-dataset Evaluation (Mueller et al. 2022) 20
14.2 Fusão Wavelet + Cepstral 20
14.3 Deep Learning em Sub-bandas 20

III APÊNDICES **22**

15 Checklist Geral dos 12 Meses **23**

16 Glossário **24**

17 Bibliografia Recomendada **26**

18 Boas Práticas Transversais **27**

Parte I

MÊS 11: Submissão do Artigo e Congelamento da Biblioteca

Capítulo 1

O que o plano exige (Mês 11)

Neste mês, o objetivo principal é submeter o artigo e congelar a biblioteca de extração de características (o seu *artifact*).

Atividades principais:

- Revisão final e submissão do artigo.
- Congelar a biblioteca e o repositório.
- Adicionar as licenças adequadas (`LICENSE`) e as formas de citação (`CITATION.cff`).
- Opcional: geração de DOI via Zenodo.
- Garantir que o código respeita as licenças originais dos corpora de áudio, mantendo a restrição ao uso acadêmico, adequando-se à LGPD.

Marco / Entrega

Marco 11: Artigo submetido + biblioteca/repositório público, acessível e reprodutível.

Capítulo 2

Revisão do Artigo

Antes da submissão, é importante passar por um checklist.

2.1 Checklist de Revisão

- **Título:** Está claro, atrativo e reflete a solução?
- **Resumo (Abstract):** Contém o problema, solução (WPT), dataset (ex: ASVspoof) e o melhor resultado (ex: EER).
- **Metodologia:** A arquitetura e os parâmetros estão bem descritos, garantindo a reprodutibilidade?
- **Resultados:** Tabelas e gráficos têm legendas, *labels* e análises comparativas?
- **Conclusão:** Relaciona aos objetivos iniciais do projeto PIBITI?

2.2 Automatizando Tabelas LaTeX com Python

Às vezes os resultados estão em `csv` ou `pandas.DataFrame`. Podemos automatizar a geração de código LaTeX para as tabelas.

Listing 2.1: Gerando tabela LaTeX a partir do Pandas

```
import pandas as pd

# Resultados ficticios
dados = {
    'Modelo': ['GMM', 'XGBoost', 'SVM'],
    'Corpus': ['ASVspoof', 'ASVspoof', 'ASVspoof'],
    'EER (%)': [4.5, 2.1, 5.8],
    'min-DCF': [0.12, 0.05, 0.19]
}
df = pd.DataFrame(dados)

# Gera codigo LaTeX
tabela_latex = df.to_latex(index=False, caption="Resultados
   preliminares por classificador", label="tab:resultados_class")
print(tabela_latex)
```

Listing 2.2: Script para formatar números na tabela LaTeX

```
# Ajustar numero de casas decimais antes da conversao
df_formatado = df.copy()
df_formatado['EER (%)'] = df_formatado['EER (%)'].apply(lambda x: f"{x:.2f}\%")
df_formatado['min-DCF'] = df_formatado['min-DCF'].apply(lambda x: f"{x:.4f}")

tabela_latex_formatada = df_formatado.to_latex(index=False)
print(tabela_latex_formatada)
```

Capítulo 3

Eventos Alvo

Para projetos de Iniciação Científica, existem bons locais para submeter os artigos:

- **ENIAC (Encontro Nacional de Inteligência Artificial e Computacional):** Reúne trabalhos de IA do Brasil. Prazos tipicamente no meio do ano.
- **CONIC / SEMESP:** Congresso Nacional de Iniciação Científica. Focado na graduação.
- **REIC (Revista Eletrônica de Iniciação Científica em Computação):** Revista da SBC para IC. Fluxo contínuo de submissão.

Dica

Verifique as datas exatas no site da SBC (Sociedade Brasileira de Computação) e adapte seu planejamento de acordo com o *call for papers* (CFP).

Capítulo 4

Congelar Biblioteca

"Congelar"a biblioteca significa lançar uma versão final estável do seu código. Para isso, utilizamos o Git.

Listing 4.1: Adicionando uma Tag no Git

```
# Certifique-se de estar na branch principal e atualizado
git checkout main
git pull

# Adicionar tag (ex: v1.0.0)
git tag -a v1.0.0 -m "Versao 1.0.0: Submissao final do PIBITI"

# Enviar a tag para o Github
git push origin v1.0.0
```

Listing 4.2: Listar e checar as tags

```
git tag -l
git show v1.0.0
```

Listing 4.3: Gerando um arquivo de versão no Python

```
echo "__version__ = '1.0.0'" > wpt_audio_spoof/__version__.py
```

Capítulo 5

LICENSE

5.1 Licença MIT

A Licença MIT é permissiva e excelente para projetos acadêmicos. Você deve incluir um arquivo `LICENSE` na raiz do repositório.

Listing 5.1: Exemplo do arquivo LICENSE (MIT)

```
MIT License

Copyright (c) 2027 Thiago Schiavi

Permission is hereby granted, free of charge, to any person
   obtaining a copy
of this software and associated documentation files (the "Software"
   ), to deal
in the Software without restriction, including without limitation
   the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/
   or sell
...
```

Listing 5.2: Adicionando o header de licença no código Python

```
1 # Em cada arquivo principal, voce pode colocar no inicio:
2 # Copyright (c) 2027 Thiago Schiavi
3 # Este codigo esta licenciado sob a MIT License.
```

Capítulo 6

CITATION.cff

Para garantir que você seja citado corretamente, crie um arquivo `CITATION.cff`.

Listing 6.1: Exemplo de CITATION.cff

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
authors:
  - family-names: "Schiavi"
    given-names: "Thiago"
title: "WPT Audio Deepfake Detection Library"
version: 1.0.0
date-released: 2027-08-01
url: "https://github.com/thiagoschiavi/wpt_audio_spoof"
```

Capítulo 7

DOI via Zenodo

O Zenodo integra-se com o GitHub para gerar um DOI (*Digital Object Identifier*) às suas versões lançadas.

Passo a passo:

1. Faça login em `https://zenodo.org` (com sua conta do GitHub).
2. Vá na opção "GitHub"no menu e ative o repositório do PIBITI.
3. Crie uma nova *Release* (`git tag`) no GitHub.
4. O Zenodo baixará o `.zip` automaticamente e emitirá um DOI!

Listing 7.1: Exemplo de chamada via API (Opcional - Zenodo API Dummy)

```
import requests
import json

# NOTA: O fluxo manual no site eh mais recomendavel para iniciantes.
# Este eh apenas um exemplo de como a API REST do Zenodo recebe requests.
headers = {"Content-Type": "application/json"}
dados = {
    "metadata": {
        "title": "WPT Audio Deepfake Detection",
        "upload_type": "software",
        "creators": [{"name": "Schiavi, Thiago", "affiliation": "Instituicao"}]
    }
}
# requests.post("https://zenodo.org/api/deposit/depositions", json=dados, ...)
```

Capítulo 8

Licenças e LGPD

Trabalhar com áudios (*datasets*) e modelos de IA requer cuidados legais e éticos.

Atenção

Muitos corpora de áudio (como partes do LibriSpeech ou VCTK) são apenas para uso não comercial. Você **NÃO DEVE** redistribuir os arquivos `.wav` no seu repositório. Forneça apenas os scripts para extração, os resultados processados em tabelas e os *links* oficias de download do corpus original.

Listing 8.1: Exemplo de aviso no README sobre os dados

```
# README.md
## Dataset
Este repositorio NAO inclui os arquivos de audio originais.
Para utilizar o sistema, baixe o corpus oficial no site do ASVspoof
(Academic Use Only) e extraia na pasta ‘data/‘.
```

Listing 8.2: .gitignore para não vazar áudios

```
# No seu arquivo .gitignore:
data/*.wav
data/*.flac
data/**/*.wav
```

Parte II

MÊS 12: Relatório Final e Apresentação

Capítulo 9

O que o plano exige (Mês 12)

Objetivo: Consolidar tudo em um relatório final e preparar a apresentação.

Atividades:

- Escrever o relatório final.
- Preparar a apresentação / poster.
- Entregas (código + relatorio).

Marco / Entrega

Marco 12: Relatório final entregue + apresentação pronta + biblioteca de *features wavelet* publicada.

Capítulo 10

Relatório Final

10.1 Estrutura / Template do Relatório

1. **Resumo e Introdução:** Motivação (Deepfakes), objetivo (PIBITI) e inovação (WPT).
2. **Objetivos Alcançados:** Comparar com o plano original.
3. **Metodologia:** Como a extração foi feita. Explicar a Wavelet, decomposição e sub-bandas.
4. **Resultados:** Dividido por Corpus, nível, família da wavelet.
5. **Análises:** Custo de CPU, Análise de Sub-bandas (quais importam mais?).
6. **Limitações:** Onde o sistema falha.
7. **Trabalhos Futuros:** Cross-dataset, deep learning, etc.

Capítulo 11

Compilar Resultados Finais

Listing 11.1: Script de consolidação de resultados

```
import glob
import pandas as pd
import os

def compilar_csvs(diretorio):
    arquivos = glob.glob(f"{diretorio}/*.csv")
    lista_df = []
    for f in arquivos:
        try:
            df = pd.read_csv(f)
            df['origem'] = os.path.basename(f)
            lista_df.append(df)
        except Exception as e:
            print(f"Erro no {f}: {e}")
    return pd.concat(lista_df, ignore_index=True) if lista_df else
       pd.DataFrame()

# uso:
df_final = compilar_csvs('./resultados_finais/')
# df_final.to_csv('resumo_total.csv')
```

Listing 11.2: Script para pegar o melhor resultado por corpus

```
def melhores_resultados(df, metrica='EER'):
    # Assumindo que EER menor eh melhor
    # Agrupamos por Corpus e pegamos o indice do menor valor
    idx_melhores = df.groupby('Corpus')[metrica].idxmin()
    return df.loc[idx_melhores]

# print(melhores_resultados(df_final))
```

Listing 11.3: Analise de custo de CPU (Exemplo de metricas)

```
def tabela_custo_cpu(df):
    if 'tempo_inferencia_ms' in df.columns:
        resumo = df.groupby('Modelo')['tempo_inferencia_ms'].mean()
        return resumo
```

```
5       return None
```

Capítulo 12

Apresentação / Poster

Estrutura sugerida para 10 slides:

1. Título e Apresentação (1 slide)
2. O Problema (1 slide - áudios gerados por IA, fraudes)
3. A Solução / Objetivo (1 slide)
4. O que é WPT? (1 slide - intuição)
5. O Pipeline de Extração (1 slide)
6. Metodologia Experimental / Datasets (1 slide)
7. Resultados - Acurácia/EER (1 slide)
8. Resultados - Sub-bandas e Eficiência (1 slide)
9. Conclusão (1 slide)
10. Repositório / Agradecimentos (1 slide)

Capítulo 13

Entregas Finais

Checklist de Entregas:

Artigo Submetido.

Relatório Final em PDF enviado no sistema da agência.

Código no GitHub (público, com README completo e LICENSE).

Apresentação / Poster.

Dados estatísticos (`resultados_finais.csv`) guardados em um drive seguro.

Capítulo 14

Trabalhos Futuros

O projeto não acaba na entrega do relatório! Algumas ideias de expansão:

14.1 Cross-dataset Evaluation (Mueller et al. 2022)

Avaliar se o modelo treinado em ASVspoof consegue detectar ataques gerados no corpus WaveFake.

Listing 14.1: Logica basica de Cross-dataset

```
def testar_cross_dataset(modelo_treinado, dados_teste_novo_corpus):
    predicoes = modelo_treinado.predict(dados_teste_novo_corpus['X'])
    eer = calcular_eer(dados_teste_novo_corpus['y'], predicoes)
    print(f"EER no corpus desconhecido: {eer:.2f}%")
    return eer
```

14.2 Fusão Wavelet + Cepstral

Combinar *features* WPT com LFCCs ou MFCCs.

Listing 14.2: Fusão simples em nível de feature

```
import numpy as np

def fundir_features(features_wpt, features_lfcc):
    # Concatena horizontalmente
    return np.hstack((features_wpt, features_lfcc))
```

14.3 Deep Learning em Sub-bandas

Usar Redes Neurais Convolucionais 1D (CNNs) diretamente nos coeficientes das sub-bandas.

Listing 14.3: Modelo Dummy CNN 1D para sub-bandas (PyTorch)

```
import torch.nn as nn

```

```
class WaveletSubbandCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv1d(in_channels=1, out_channels=16,
           kernel_size=3)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool1d(2)
        self.fc = nn.Linear(16 * 10, 2) # Exemplo 10 dim

    def forward(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x
```

Parte III

APÊNDICES

Capítulo 15

Checklist Geral dos 12 Meses

M1: Estudo teórico e preparo de ambiente.

M2: Entendimento sobre WPT.

M3: Prototipação de features e energia RMS.

M4: Finalização do extrator core (WPT) base.

M5: Expansão para sub-bandas, médias e desvios.

M6: Entendimento de corpora (ASVspoof/WaveFake).

M7: Primeiros treinamentos base (SVM, GMM, RF).

M8: Análise de hiperparâmetros do WPT.

M9: Extração massiva e salva-guarda (HDF5).

M10: Escrita do artigo, cross-validation e análise fina.

M11: Submissão do artigo, licenças, congelamento (tags/DOI).

M12: Relatório final e encerramento.

Capítulo 16

Glossário

amplitude: Valor máximo de oscilação de uma onda de áudio.

bonafide: Áudio real, humano legítimo, não sintetizado.

corpus: (Plural: corpora) Conjunto estruturado de dados (neste caso, áudios anotados).

DWT: Discrete Wavelet Transform. Decompõe o sinal, mas é menos simétrica que WPT.

EER: Equal Error Rate. Ponto onde FAR (Falsos Aceitos) é igual a FRR (Falsos Rejeitados).

envelope: Contorno de variação da amplitude do sinal de áudio.

FAR: False Acceptance Rate. Proporção de spoof detectada como real.

feature: Característica ou atributo extraído matematicamente do sinal.

FRR: False Rejection Rate. Proporção de áudio real detectada como spoof.

GMM: Gaussian Mixture Model. Classificador clássico baseado em distribuições gaussianas.

Hilbert: Transformada util para obter a energia instantânea e o envelope analítico do sinal.

in-domain: Avaliação feita dentro do mesmo corpus em que o modelo foi treinado.

min-DCF: Minimum Detection Cost Function. Outra métrica rigorosa de erro.

mono: Áudio de apenas 1 canal.

Nyquist: A frequência máxima representável é a metade da taxa de amostragem.

padding: Adicionar zeros nas bordas do sinal para preencher um tamanho específico.

RMS: Root Mean Square. Medida clássica de energia do sinal.

sample rate: Quantidade de amostras por segundo. (ex: 16 kHz).

spoof: Áudio falsificado ou sintetizado maliciosamente.

sub-banda: Intervalos frequenciais gerados pela decomposição em níveis.

TTS: Text-to-Speech. Síntese de texto para voz.

VC: Voice Conversion. Conversão da voz de um locutor na de outro.

wavelet: Função matemática de duração finita usada para analisar sinais temporais.

wavelet-mae: Coeficiente de erro absoluto médio em uma decomposição wavelet.

WPT: Wavelet Packet Transform. Extensão da DWT que decompõe tanto a aproximação quanto os detalhes em cada nível.

XGBoost: Algoritmo robusto de *Gradient Boosting* em *trees*.

ZCR: Zero Crossing Rate. Taxa de cruzamentos por zero do sinal temporal.

Capítulo 17

Bibliografia Recomendada

- **Wavelets:** Daubechies, I. (1992). *Ten Lectures on Wavelets.*
- **Anti-spoofing:** Sahidullah, M., et al. (2015). *A comparison of features for synthetic speech detection.*
- **Corpora:** Yamagishi, J., et al. (2019). *ASVspoof 2019: Future Horizons in Spoofed and Fake Audio Detection.*
- **ML:** Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning.*

Capítulo 18

Boas Práticas Transversais

Listing 18.1: 1. Controle de Seeds

```python
import numpy as np
np.random.seed(42) # garante reproducibilidade
```

Listing 18.2: 2. Fechar arquivos HDF5

```python
import h5py
# Use com 'with' para gerenciar contexto automaticamente
def ler_hdf5(caminho):
    with h5py.File(caminho, 'r') as f:
        return f['dataset'][:]
```

Listing 18.3: 3. Assertivas no Codigo

```python
def processar_audio(sinal):
    assert len(sinal) > 0, "Sinal vazio"
    return sinal * 2
```

Listing 18.4: 4. Logging e Nao Print

```python
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Extracao concluida para o arquivo 1")
```

Listing 18.5: 5. Tipagem

```python
def calcular_energia(sinal: list[float]) -> float:
    return sum([x**2 for x in sinal])
```

Listing 18.6: 6. Profiling Simples

```python
import time
inicio = time.time()
# codigo
fim = time.time()
print(f"Tempo decorrido: {fim-inicio} seg")
```

Listing 18.7: 7. Requirements.txt limpo

```
# Nao de pip freeze direto. Filtre os pacotes importantes.
numpy==1.23.5
pandas==1.5.2
scipy==1.9.3
scikit-learn==1.2.0
PyWavelets==1.4.1
```

Listing 18.8: 8. Evitar loops do Python com Numpy

```
import numpy as np
arr = np.array([1,2,3,4,5])
# Lento: [x*2 for x in arr]
# Rapido:
arr_novo = arr * 2
```

Listing 18.9: 9. Pathlib em vez de os.path

```
from pathlib import Path
caminho = Path("data") / "audio.wav"
print(caminho.exists())
```

Listing 18.10: 10. Docstrings

```
def calcular_eer(y_true, y_score):
    """
    Calcula o Equal Error Rate (EER).

    Args:
        y_true (np.array): Labels reais.
        y_score (np.array): Probabilidades / scores preditos.

    Returns:
        float: O valor do EER.
    """
    pass
```